Masked PPO with Constraint-Aware Features for Learning to Solve Hitori

Han Deng
Department of Information Engineering
CUHK
Shatin, Hong Kong
1155254760@link.cuhk.edu.hk

Abstract

Hitori is a highly structured logic puzzle whose local duplicate-removal and adjacency rules interact with a global connectivity constraint, making it a challenging combinatorial decision problem. We study Hitori from a reinforcement-learning perspective and present a practical solving pipeline based on Maskable Proximal Policy Optimization (PPO). Our approach combines invalid-action masking, a unified variable-size padded formulation, a same-resolution U-Net policy with constraint-aware feature channels, shaped reward, and a cold-start supervised fine-tuning (SFT) stage from symbolic-solver trajectories. Together, these components reduce the effective branching factor, improve optimization stability, and enable one policy to train across multiple board sizes. Experiments on boards of size 4–20 show that SFT provides a strong initialization signal and exhibits a clear power-law-like scaling trend with dataset size, mixed-size training consistently outperforms size-specific training, and both reward shaping and SFT are especially important on larger boards. The U-Net architecture also substantially outperforms simpler CNN and flattened baselines, highlighting the importance of preserving spatial alignment between cells and actions. Compared with a time-limited symbolic solver, the learned policy is less accurate on small boards but scales more gracefully, and the full SFT+RL pipeline overtakes the symbolic baseline on sufficiently large instances under a fixed inference-time budget. These results suggest that constraint-aware RL can learn meaningful Hitori-solving behavior and serve as a promising amortized alternative to search on larger puzzles.

1 Introduction

Hitori is a Japanese pencil-and-paper logic puzzle popularized by Nikoli. At a high level, it resembles Sudoku in that the player reasons over a number grid under strict consistency rules, but its objective is different: instead of filling empty cells, the solver must black out a subset of cells so that three constraints are satisfied simultaneously. First, no number may appear more than once among the *unshaded* cells of any row or column. Second, black cells may not share an edge. Third, all remaining white cells must form a single connected component [1]. These simple local rules interact with a global connectivity constraint, making Hitori a compact but highly structured combinatorial decision problem.

From a computational perspective, Hitori is widely regarded as a hard puzzle family; prior work describes the problem as NP-complete, and therefore NP-hard in the standard optimization sense considered in algorithm design [2, 3]. This hardness is not merely of theoretical interest. In practical solvers, the main difficulty is that legal-looking local moves can easily destroy future feasibility, especially because duplicate-removal, adjacency avoidance, and global connectivity must all hold at the end of the episode. As a result, exact approaches typically rely on explicit search or on reductions

Preprint.

to mature combinatorial solving frameworks. Existing Hitori solvers have been formulated using custom backtracking and rule-based deduction, Boolean satisfiability (SAT), satisfiability modulo theories (SMT), answer set programming (ASP), and integer linear programming (ILP) [4, 5, 3, 6]. While such approaches can be very effective, they usually depend on handcrafted encodings, carefully engineered constraints, and search procedures whose computational cost grows quickly with puzzle size and structural complexity.

This report explores a different perspective: treating Hitori as a reinforcement learning problem and solving it with *masked* Proximal Policy Optimization (PPO). PPO is a widely used policy-gradient method that offers a favorable balance between simplicity, stability, and empirical performance [7]. For Hitori, however, naive reinforcement learning is a poor fit because the action space contains many invalid or clearly harmful moves. We therefore adopt invalid-action masking, a mechanism that restricts sampling to feasible actions and has been theoretically and empirically justified for policy-gradient methods [8]. More broadly, Hitori exposes several difficulties for RL at once: the action space changes with board size, rewards are naturally sparse and delayed, and successful trajectories are rare on larger boards. To address these issues, we build a constraint-aware pipeline that combines masked PPO with a unified padded representation for variable-size boards, and to extract practical insights about when action masking, reward shaping, and curriculum design help or fail in this domain.

In the remainder of the report, we present our Hitori environment, the masked PPO training setup, and a series of empirical observations from this implementation. The emphasis is on practical understanding: what makes Hitori difficult for reinforcement learning, what architectural and reward choices matter most, and what these experiments suggest about the broader relationship between structured logic puzzles and modern RL methods.

2 Method

2.1 Environment

Our implementation builds on the open-source `hitori-gym` environment, which provides a Gym-style formulation of Hitori with dynamic legality checking and puzzle generation [9]. For an $n \times n$ board, the original observation is a dictionary

$$o_t = \{\texttt{game\_grid}, \texttt{shaded}\}, \qquad \texttt{game\_grid} \in \{1, \dots, n\}^{n \times n}, \quad \texttt{shaded} \in \{0, 1\}^{n \times n}, \tag{1}$$

and the action space is discrete,

$$a_t \in \{0, 1, \dots, n^2 - 1\}, \tag{2}$$

where each action selects one cell to shade in row-major order.

A key design choice is *action masking* rather than penalizing illegal moves after the fact. At each state, the environment computes a binary mask

$$m_t \in \{0, 1\}^{n^2}, \tag{3}$$

where $m_t(a) = 1$ indicates that action a is legal. The mask excludes cells that are already shaded, adjacent to a shaded cell, articulation points whose removal would disconnect the remaining white-cell graph, or already unique in both row and column. In particular, the articulation-point check provides an efficient graph-theoretic pre-test for the global connectivity constraint.

The original repository uses a sparse reward,

$$r_t^{\text{orig}} = \begin{cases} +1.0, & \text{if the puzzle is solved,} \\ -1.0, & \text{if the agent gets stuck with no valid moves,} \\ -0.01, & \text{otherwise.} \end{cases} \tag{4}$$

In our project, we retain this sparse setting as the basic training configuration, but convert the environment to a unified padded interface for fixed-size policies and mixed-size training. Given a maximum board size `maxn`, each board is placed in the upper-left corner of a `maxn` $\times$ `maxn` canvas, with the remaining entries padded by zero:

$$\tilde{o}_t = \{\widetilde{\texttt{game\_grid}}, \widetilde{\texttt{shaded}}\}, \qquad \mathcal{A} = \{0, 1, \dots, \texttt{maxn}^2 - 1\}. \tag{5}$$

The action mask is padded in the same way, so all padded positions always have mask value 0. This gives a single observation space and action space shared across board sizes, allowing one policy architecture to train on either fixed-size or mixed-size puzzles.

2.2 Maskable PPO

We train the agent with Maskable Proximal Policy Optimization (MaskablePPO), a masked-action extension of PPO [7, 8]. Let $\pi_\theta(a \mid s)$ and $V_\phi(s)$ denote the policy and value function. The PPO objective is

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}\big(r_t(\theta), 1-\epsilon, 1+\epsilon\big)\hat{A}_t\Big)\Big], \qquad r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}. \tag{6}$$

We estimate advantages with generalized advantage estimation (GAE),

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \qquad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \tag{7}$$

and train the critic by squared error to the return target $\hat{R}_t$:

$$\mathcal{L}^V(\phi) = \mathbb{E}_t\Big[(V_\phi(s_t) - \hat{R}_t)^2\Big]. \tag{8}$$

An entropy bonus is added for exploration, giving the overall objective

$$\mathcal{L}(\theta, \phi) = \mathcal{L}^{\text{PPO}}(\theta) - c_v\mathcal{L}^V(\phi) + c_e\,\mathbb{E}_t[\mathcal{H}(\pi_\theta(\cdot \mid s_t))], \tag{9}$$

where c_v and c_e are the value and entropy coefficients.

The key difference from vanilla PPO is that probabilities are normalized only over valid actions. Given a binary mask $m(s) \in \{0,1\}^{|\mathcal{A}|}$ and logits $z_\theta(s) \in \mathbb{R}^{|\mathcal{A}|}$, the masked policy is

$$\pi_\theta^{\text{mask}}(a \mid s) = \frac{\exp(z_\theta(s,a))\, m(s,a)}{\sum_{a'} \exp(z_\theta(s,a'))\, m(s,a')}, \tag{10}$$

which is equivalent to assigning invalid actions logit $-\infty$ before softmax. For Hitori, this is especially useful because many actions are known to be infeasible from the current state. Masking therefore restricts exploration to the feasible set, reduces the effective branching factor, and improves training stability.

2.3 Model and Feature Construction

We design the policy to operate at the same spatial resolution as the board, since each Hitori action corresponds to selecting exactly one cell. Rather than flattening the board and using an MLP, we use a convolutional architecture that preserves two-dimensional structure throughout. This is well suited to Hitori, where duplicate patterns, adjacency constraints, and connectivity all have strong spatial regularity.

Each padded board state is converted into a 9-channel feature tensor,

$$x_t \in \mathbb{R}^{9\times \texttt{maxn}\times \texttt{maxn}}, \tag{11}$$

whose channels encode: normalized puzzle value, shaded indicator, row duplicate pressure, column duplicate pressure, adjacent shaded density, valid-board mask, normalized row coordinate, normalized column coordinate, and board-size ratio. The duplicate-pressure and adjacency channels expose useful constraint information while leaving the final decision policy to learning.

Our feature extractor is a compact U-Net backbone [10] with three encoder stages, one bottleneck, and three decoder stages with skip connections:

$$\begin{aligned}
e_1 &= \text{ConvBlock}(x_t),\\
e_2 &= \text{ConvBlock}(\text{Pool}(e_1)),\\
e_3 &= \text{ConvBlock}(\text{Pool}(e_2)),\\
b &= \text{ConvBlock}(\text{Pool}(e_3)),\\
d_1 &= \text{UpBlock}(b, e_3),\\
d_2 &= \text{UpBlock}(d_1, e_2),\\
d_3 &= \text{UpBlock}(d_2, e_1),\\
f_t &= \text{Conv}_{1\times1}(d_3).
\end{aligned} \tag{12}$$

Each `ConvBlock` contains two 3×3 convolutions with a residual connection, GroupNorm, and SiLU activation, while each `UpBlock` upsamples by bilinear interpolation, concatenates the corresponding skip feature, and applies another convolutional block. Although the implementation supports axial attention in the bottleneck, our final experiments disable it because it consistently hurt performance. The reported model is therefore a purely convolutional U-Net.

The backbone outputs

$$f_t \in \mathbb{R}^{C \times \texttt{maxn} \times \texttt{maxn}}, \qquad C = 64. \tag{13}$$

The actor applies a 1×1 convolution to produce one logit per cell,

$$\ell_t = \mathrm{Conv}_{1\times1}^{\text{actor}}(f_t) \in \mathbb{R}^{1 \times \texttt{maxn} \times \texttt{maxn}}, \tag{14}$$

which is then flattened into $\texttt{maxn}^2$ action logits and filtered by the action mask. This preserves a direct correspondence between board locations and action indices.

The critic uses a separate 1×1 convolution followed by masked spatial average pooling:

$$u_t = \mathrm{Conv}_{1\times1}^{\text{value}}(f_t), \qquad \bar{u}_t = \frac{\sum_{i,j} u_t(:,i,j)\, b_t(i,j)}{\sum_{i,j} b_t(i,j)}, \qquad V_\phi(s_t) = W\bar{u}_t + c, \tag{15}$$

where $b_t(i,j) \in \{0,1\}$ indicates whether location (i,j) belongs to the real board rather than padding. This allows one shared critic to handle multiple board sizes while ignoring padded cells.

Overall, the model combines three inductive biases that are especially suitable for Hitori: spatially aligned per-cell actions, constraint-aware handcrafted features, and a same-resolution convolutional backbone.

2.4 Variable-Size Adaptation

A practical challenge in Hitori reinforcement learning is that the natural action space depends on board size: an $n \times n$ puzzle has n^2 cell-selection actions. A naive solution is to train a separate policy for each size, but this is wasteful because the underlying rules are identical across sizes: row/column duplicate removal, no adjacent black cells, and connectivity of white cells. A unified policy should therefore be able to share knowledge across instance scales.

To achieve this, we use a fixed-size padded representation with $\texttt{maxn} = 20$. Any board of size $n \le \texttt{maxn}$ is embedded into the upper-left corner of a $\texttt{maxn} \times \texttt{maxn}$ canvas:

$$\widetilde{G}_{i,j} = \begin{cases} G_{i,j}, & 1 \le i,j \le n, \\ 0, & \text{otherwise}, \end{cases} \qquad \widetilde{S}_{i,j} = \begin{cases} S_{i,j}, & 1 \le i,j \le n, \\ 0, & \text{otherwise}, \end{cases} \qquad B_{i,j} = \begin{cases} 1, & 1 \le i,j \le n, \\ 0, & \text{otherwise}, \end{cases} \tag{16}$$

where G is the puzzle grid, S is the shaded-cell indicator, and B is a binary mask marking valid board positions. The same padding is applied to all derived feature channels.

This gives a shared observation space and action space for all environments,

$$\tilde{o}_t \in \mathbb{R}^{9 \times \texttt{maxn} \times \texttt{maxn}}, \qquad \mathcal{A} = \{0, 1, \ldots, \texttt{maxn}^2 - 1\}. \tag{17}$$

For boards with $n < \texttt{maxn}$, all padded positions are marked invalid by the action mask, i.e.,

$$m^{(n)}(s,a) = 0 \quad \text{for padded actions } a, \tag{18}$$

so the policy always outputs a fixed-length logit vector while decisions are restricted to the valid $n \times n$ region.

This representation allows a single model to train on mixed board sizes, supports cross-size transfer, and keeps the network architecture unchanged across settings. It also enables us to study two questions experimentally: length extrapolation from smaller to larger boards, and the trade-off between mixed-size training and size-specific training.

2.5 Shaped Reward

The original sparse reward is often too weak for Hitori, especially on larger boards: the agent receives meaningful feedback only at success or failure, making credit assignment difficult over long horizons. We therefore add a constraint-aware shaped reward on top of `hitori-gym` [9].

To measure progress toward a valid solution, we track three state statistics:

$$\mathrm{dup}(s), \qquad \mathrm{adj}(s), \qquad \mathrm{comp}(s), \tag{19}$$

which denote respectively the number of remaining duplicate violations among unshaded cells, the number of adjacent shaded-cell conflicts, and the excess number of connected components in the unshaded region. A solved board satisfies all three at zero.

For a transition (s_t, a_t, s_{t+1}), define

$$\Delta_{\mathrm{dup}} = \mathrm{dup}(s_t) - \mathrm{dup}(s_{t+1}), \qquad \Delta_{\mathrm{adj}} = \mathrm{adj}(s_t) - \mathrm{adj}(s_{t+1}), \qquad \Delta_{\mathrm{comp}} = \mathrm{comp}(s_t) - \mathrm{comp}(s_{t+1}), \tag{20}$$

so positive values indicate improvement. The dense shaping term is then

$$r_t^{\mathrm{dense}} = \lambda_{\mathrm{dense}} \, \mathrm{Scale}(\beta_{\mathrm{step}} + \alpha_{\mathrm{dup}}\Delta_{\mathrm{dup}} + \alpha_{\mathrm{adj}}\Delta_{\mathrm{adj}} + \alpha_{\mathrm{comp}}\Delta_{\mathrm{comp}}, \, n) \, , \tag{21}$$

where $\beta_{\mathrm{step}} < 0$ is a step penalty, the α coefficients weight the three progress terms, and n is the true board size. The normalization function is

$$\mathrm{Scale}(r, n) = \begin{cases} r, & \text{if } \texttt{shaped\_reward\_norm} = \texttt{none}, \\ r/n^2, & \text{if } \texttt{shaped\_reward\_norm} = \texttt{inv\_board\_area}. \end{cases} \tag{22}$$

A terminal bonus or penalty is added separately:

$$r_t^{\mathrm{terminal}} = \begin{cases} \beta_{\mathrm{success}}, & \text{if the episode ends in a solved state,} \\ \beta_{\mathrm{fail}}, & \text{if the episode ends in failure,} \\ 0, & \text{otherwise,} \end{cases} \qquad r_t = r_t^{\mathrm{dense}} + r_t^{\mathrm{terminal}}. \tag{23}$$

In the default setting,

$$\alpha_{\mathrm{dup}} = 1.0, \quad \alpha_{\mathrm{adj}} = 0.5, \quad \alpha_{\mathrm{comp}} = 1.5, \quad \beta_{\mathrm{success}} = 10.0, \quad \beta_{\mathrm{fail}} = -10.0, \quad \beta_{\mathrm{step}} = -0.05. \tag{24}$$

We use $\texttt{shaped\_reward\_norm} = \texttt{none}$ and $\lambda_{\mathrm{dense}} = 1.0$ by default; if area normalization is enabled, the code uses $\lambda_{\mathrm{dense}} = 15.0$ unless manually overridden.

This shaping rule is a practical heuristic rather than a strictly potential-based reward in the sense of [11]. Its purpose is to provide denser and more interpretable supervision, and in practice it improves optimization stability substantially over the sparse reward, particularly on larger boards.

2.6 Cold-Start Supervised Fine-Tuning

Even with action masking and shaped reward, PPO is difficult to optimize from scratch in Hitori, especially on larger boards where successful trajectories are rare. Early in training, the actor distributes probability almost arbitrarily over legal actions, while the critic produces noisy value estimates; because both share the same backbone, unstable critic gradients can disrupt representation learning. To reduce this cold-start problem, we add a supervised fine-tuning (SFT) stage before PPO.

We first generate supervision data with `generate_solver_supervision_dataset.py`. Puzzle instances are sampled from the same generator as the RL environment and solved by a symbolic solver derived from `hitori-solver` [12, 9]. We use 128 parallel workers for board sizes 4 to 20, with a 60-second time limit per puzzle; unsolved instances are discarded. Each accepted sample provides a puzzle and its final binary solution mask

$$S^\star \in \{0, 1\}^{n \times n}, \tag{25}$$

where $S^\star_{i,j} = 1$ indicates that cell (i, j) is shaded in the solution.

Because the actor requires sequential decisions rather than a final mask, `train_sft.py` reconstructs an expert trajectory by replaying the solution under the real environment dynamics. Starting from an empty shading pattern, at each step we choose the first row-major cell that is shaded in $S^\star$, not yet selected, and currently legal under the environment mask:

$$a_t = \min_{\text{row-major}} \left\{ (i, j) \mid S^\star_{i,j} = 1, \; S_{t,i,j} = 0, \; m_t(i, j) = 1 \right\}. \tag{26}$$

This produces trajectories $\tau = \{(o_t, m_t, a_t)\}_{t=0}^{T-1}$ of padded observations, padded masks, and padded action indices. If replay becomes inconsistent with the target solution, the trajectory is discarded. The resulting dataset therefore matches the same transition and masking logic used later in PPO. We then train by masked behavioral cloning with

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\frac{1}{B}\sum_{i=1}^{B} \log \pi_\theta(a_i \mid o_i, m_i), \tag{27}$$

using a train/validation split at the puzzle level.

After SFT, the pretrained checkpoint is used only to initialize PPO parameters, not optimizer state. To further stabilize training, we apply a short critic warmup phase inside the normal PPO loop. For the U-Net policy, the backbone, actor head, and critic intermediate head are frozen, and only the final scalar value readout is trainable:

$$\theta, \psi, \eta \text{ frozen}, \qquad w \text{ trainable}. \tag{28}$$

After several update phases, all parameters are unfrozen:

$$\theta, \psi, \eta, w \text{ all trainable}. \tag{29}$$

This two-stage initialization gives the actor a useful prior over legal Hitori moves while preventing early critic instability from immediately overwriting the pretrained representation, which substantially improves PPO optimization in mixed-size and large-board settings.

3 Experiments

3.1 Default Experimental Setup

Unless a subsection states otherwise, experiments use the following protocol.

Cold-start SFT. Expert trajectories are generated with the symbolic pipeline from Sec. 2 (128 parallel workers, 60-second time limit per puzzle; unsolved instances discarded). Default SFT trains jointly on board sizes 4–20 with 1000 expert puzzles per size, a puzzle-level validation split of 10%, learning rate 3×10^{-4}, and 10 epochs of masked behavioral cloning. We keep the checkpoint with the lowest validation negative log-likelihood. When we report standalone SFT solve rates, we evaluate on 50 held-out puzzles per size in the Hitori environment.

Maskable PPO. Default reinforcement learning runs for 4×10^6 environment timesteps at learning rate 10^{-4}. After SFT initialization, we apply critic warmup for 10^5 timesteps (Sec. 2, cold-start SFT). Rollouts use 128 parallel environments (`-n-envs 128`), horizon `-n-steps` 32, and update minibatches of size `-batch-size` 256. The padded board uses `-max-size` 20, and by default both training and evaluation draw puzzles from sizes 4–20 (`train-sizes`=`eval-sizes`= 4–20). Some subsections below report only up to size 11; in those runs we set `train-sizes` and `eval-sizes` to 4–11 instead.

Compute budgets. For ablations we shorten RL training to 2×10^6 steps for efficiency. When training or evaluating a single board size in isolation, we use 5×10^5 steps so that total compute remains comparable across size-specific runs.

3.2 Scaling Law of Cold-Start SFT

We study how cold-start supervised fine-tuning (SFT) scales with the amount of solver-generated data. Unless otherwise specified, we follow the default SFT procedure in Sec. 3.1. In this experiment, we vary the per-size SFT budget from $N = 20$ to $N = 1000$ and measure solve rate on held-out puzzles, then average across board sizes and analyze the trend in log-log space.

Figure 1 shows the results. The dashed black curve is the mean solve rate across all sizes, and the colored curves correspond to individual sizes. In log-log coordinates, the mean trend is well fit by

$$\log_{10} \max(r, 10^{-6}) = \beta \log_{10} N + \alpha, \tag{30}$$

with

$$\beta = 0.183, \qquad \alpha = -0.966, \qquad R^2_{\log} = 0.964. \tag{31}$$

Thus, over the explored range, SFT exhibits a fairly clean power-law-like improvement: more supervision yields higher average solve rate in a predictable way.

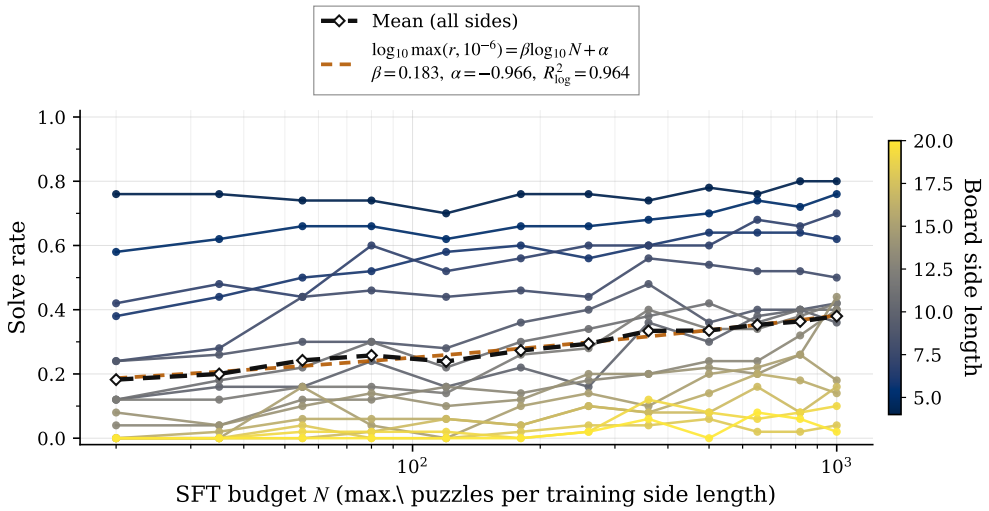

Figure 1: Scaling behavior of cold-start SFT. One policy is jointly trained on sizes 4–20 with progressively larger expert datasets. Colored curves show solve rates for individual board sizes, and the dashed black curve shows the mean solve rate across all sizes. In log-log space, the mean trend is well approximated by a linear fit.

However, this trend should not be over-interpreted. On small boards such as $n = 4$ and $n = 5$, performance already appears close to saturation, suggesting that imitation alone cannot fully solve the task. On larger boards, especially near $n = 19$ and $n = 20$, improvements are much slower. The scaling law therefore indicates not unlimited gains, but diminishing practical returns: each additional improvement requires increasingly more data and compute.

A second limitation is expert-data generation itself. Table 1 reports the fraction of random puzzles solved by the symbolic solver within 10 seconds. The success rate remains near perfect up to about size 12, but then drops rapidly: only 13% of size-20 puzzles are solved within the limit, and beyond size 24 the solver produces essentially no usable supervision. Hence, scaling SFT to larger boards is constrained not only by training cost, but also by the difficulty of obtaining expert trajectories.

Table 1: Feasibility of generating expert solutions under a 10-second solver budget. Each board size is tested on 100 instances.

Sizes 4–14			Sizes 15–25		
size	solved	rate	size	solved	rate
4	100	1.0000	15	71	0.7100
5	100	1.0000	16	54	0.5400
6	100	1.0000	17	40	0.4000
7	100	1.0000	18	38	0.3800
8	100	1.0000	19	20	0.2000
9	100	1.0000	20	13	0.1300
10	100	1.0000	21	4	0.0400
11	100	1.0000	22	3	0.0300
12	99	0.9900	23	5	0.0500
13	97	0.9700	24	0	0.0000
14	87	0.8700	25	0	0.0000
	Overall (2200 trials)		1331		0.6050

Overall, SFT is highly effective as a cold-start mechanism: it gives the policy useful priors over legal Hitori moves and improves steadily with more supervision. But it is not a complete solution. On easy boards it saturates, and on hard boards it scales slowly while relying on increasingly expensive symbolic-solver compute. We therefore view SFT not as a replacement for RL, but as an initialization stage that makes subsequent reinforcement learning substantially more effective.

3.3 Mixed-Size Training vs. Size-Specific Training

We next compare joint training across board sizes with training a separate policy for each size. Since Hitori shares the same rule structure across scales, we expect mixed-size training to improve data efficiency through cross-size transfer. To test this, we evaluate two strategies on sizes $n = 4$ to 11.

In the *mixed-size* setting, one model is trained on the union of sizes 4–11 using the unified padded representation. In the *size-specific* setting, a separate model is trained for each fixed size with 5×10^5 timesteps per size (Sec. 3.1). The mixed-size run uses the default joint budget of 4×10^6 timesteps. All runs use shaped reward and no cold-start SFT, so the comparison isolates the effect of joint training.

Table 2 shows the results. The mixed-size model outperforms the size-specific baseline on every size from 5 to 11, often by a substantial margin; only at $n = 4$ is the single-size model slightly better, and the difference is negligible.

Table 2: Solve rate comparison between mixed-size training and size-specific training. All runs use shaped reward and no SFT.

Board size (n)	Mixed training on 4–11	Size-specific training
4	0.765	0.775
5	0.760	0.725
6	0.720	0.530
7	0.575	0.360
8	0.495	0.280
9	0.400	0.275
10	0.325	0.200
11	0.255	0.140

The advantage of mixed-size training grows with board size, which suggests that the policy benefits from structural regularities shared across scales. Smaller boards provide easier and denser learning signals for core concepts such as duplicate elimination, adjacency avoidance, and connectivity preservation, and these transfer to larger boards where exploration is harder. Overall, this experiment supports unified variable-size training not only as an architectural convenience, but also as a clear empirical improvement. We therefore use mixed-size training as the default setting in the remaining experiments.

3.4 Length Extrapolation

We next study *length extrapolation*: whether a policy trained only on smaller boards can generalize to larger unseen sizes. All experiments in this subsection use shaped reward and no cold-start SFT, so the results reflect reinforcement learning alone. Each model trains for 4×10^6 timesteps (Sec. 3.1) on sizes 4 through m, with $m \in \{4, \ldots, 8\}$, and we evaluate on test sizes 4 through 15.

Table 3 reports the solve rates. Each column corresponds to one training range, and each row gives the solve rate on test size n.

Table 3: Length extrapolation results. All models use shaped reward and no SFT. Columns indicate mixed training on board sizes 4 through the listed upper bound ("4 only" means training puzzles are restricted to $n = 4$). Rows are test sizes.

n_{test}	train 4 only	train 4–5	train 4–6	train 4–7	train 4–8
4	0.780	0.780	0.765	0.765	0.765
5	0.775	0.800	0.770	0.780	0.785
6	0.725	0.735	0.670	0.770	0.730
7	0.515	0.485	0.480	0.615	0.565
8	0.355	0.305	0.430	0.540	0.480
9	0.220	0.165	0.315	0.515	0.385
10	0.135	0.050	0.235	0.320	0.300
11	0.050	0.015	0.105	0.195	0.160
12	0.030	0.040	0.070	0.130	0.145
13	0.000	0.005	0.015	0.035	0.070
14	0.000	0.005	0.005	0.035	0.035
15	0.000	0.005	0.000	0.010	0.005

Several patterns are clear. First, extrapolation to moderately larger boards is real: even a policy trained only on size 4 achieves nontrivial performance well beyond its training range, which suggests that it learns transferable structural rules rather than size-specific patterns. Second, expanding the training range generally improves out-of-distribution performance, especially on sizes just beyond the training maximum; for example, training on 4–7 substantially improves performance on unseen sizes 8–12 relative to training on 4–6.

At the same time, the improvement is not strictly monotonic. Training on 4–8 does not uniformly dominate training on 4–7, likely because adding harder sizes also makes the joint RL problem more difficult to optimize. More importantly, long-range extrapolation remains limited: solve rates eventually collapse as board size increases, and even the best setting performs only modestly on sizes 12–15.

Overall, Hitori exhibits meaningful but local scale generalization. Mixed-size RL can transfer useful structure beyond the exact training range, but this effect weakens quickly on larger boards. Thus, architectural bias and variable-size training help, yet they are not sufficient for robust large-scale generalization on their own.

3.5 Ablation on Reward Shaping and Cold-Start SFT

We next ablate the effects of reward shaping and cold-start SFT. All models are trained in the same mixed-size setting for 2×10^6 RL timesteps (Sec. 3.1), and when SFT is enabled we use the default critic warmup of 10^5 environment steps. We compare four configurations: sparse reward without pretraining, sparse reward with SFT, shaped reward without pretraining, and shaped reward with SFT.

Table 4 reports solve rates for board sizes 4 through 11.

Table 4: Ablation on reward shaping and cold-start SFT. When SFT is used, the PPO run uses the default critic warmup of $100{,}000$ steps.

Setting	4	5	6	7	8	9	10	11
Sparse, no pretrain	0.770	0.765	0.720	0.565	0.510	0.145	0.050	0.000
Sparse + SFT	0.745	0.790	0.715	0.555	0.510	0.385	0.255	0.140
Shaped, no pretrain	0.760	0.730	0.705	0.520	0.480	0.420	0.325	0.185
Shaped + SFT	0.775	0.800	0.770	0.690	0.635	0.645	0.545	0.430

Two trends are clear. First, both SFT and reward shaping mainly help on larger boards, where plain sparse-reward PPO degrades quickly. Under sparse reward, adding SFT improves solve rate from 0.145 to 0.385 at size 9, from 0.050 to 0.255 at size 10, and from 0.000 to 0.140 at size 11, showing that SFT substantially alleviates the cold-start exploration problem. Reward shaping alone also

yields large gains on medium and large boards compared to sparse training without pretraining, and combining shaping with SFT gives the strongest configuration overall.

3.6 Model Ablation

We compare three policy architectures implemented in our codebase: a U-Net style convolutional model, a plain CNN baseline, and a structured flattened baseline. All models are trained under the same RL setup and the ablation compute budget (2×10^6 timesteps, Sec. 3.1), so the comparison isolates architectural effects.

Table 5 reports solve rates on board sizes 4 through 11.

Table 5: Model ablation across three policy architectures.

Model	4	5	6	7	8	9	10	11
U-Net	0.760	0.730	0.650	0.480	0.425	0.340	0.310	0.155
CNN	0.735	0.350	0.110	0.045	0.005	0.005	0.000	0.005
Structured	0.335	0.160	0.075	0.055	0.015	0.010	0.005	0.000

The three models mainly differ in how well they preserve spatial structure. The *Structured* baseline flattens the handcrafted per-cell features into a single vector, which discards the board geometry almost entirely. The *CNN* baseline preserves locality better through standard convolutions, but eventually flattens the board into a global representation, weakening the correspondence between cells and actions. By contrast, the *U-Net* keeps a same-resolution spatial representation throughout an encoder-decoder backbone with skip connections. Although the implementation supports optional axial attention, the model used here is a pure convolutional U-Net because attention consistently hurt performance.

The results clearly favor U-Net. It outperforms both baselines on every tested board size, with especially large margins on medium and large boards. For example, at size 8, U-Net reaches 0.425, while CNN and Structured achieve only 0.005 and 0.015. This suggests that Hitori strongly benefits from preserving spatial alignment between board cells and action logits while still aggregating larger-scale context. Overall, the ablation shows that a same-resolution convolutional architecture is far better suited to this task than either flattened features or a plain CNN followed by global flattening.

3.7 Final Comparison: Search Solver vs. SFT vs. SFT+RL

Finally, we compare three approaches on board sizes 4 through 20: a CSP-style symbolic solver under a 1-second time budget, a purely supervised policy trained from solver trajectories, and our full pipeline combining cold-start SFT with reinforcement learning. The RL stage uses the default 4×10^6-step budget and size range from Sec. 3.1. The goal is to compare not only absolute solve rate, but also how the relative strengths of symbolic search and learning change with problem size.

Table 6 reports the results.

Table 6: Final comparison among a CSP-style symbolic solver, cold-start SFT, and the full SFT+RL pipeline.

Size	CSP-style symbolic solver ($\leq$ 1s)	Cold-start SFT	SFT + RL
4	1.000	0.740	0.770
5	1.000	0.750	0.800
6	1.000	0.685	0.805
7	1.000	0.595	0.670
8	1.000	0.565	0.645
9	1.000	0.515	0.585
10	0.995	0.455	0.510
11	0.950	0.340	0.400
12	0.900	0.315	0.410
13	0.780	0.245	0.430
14	0.610	0.275	0.450
15	0.465	0.195	0.395
16	0.265	0.145	0.350
17	0.170	0.110	0.260
18	0.095	0.125	0.395
19	0.055	0.105	0.310
20	0.020	0.085	0.285

The ranking of methods changes markedly with board size. On small boards, the symbolic solver is dominant, solving nearly all instances up to about size 9 within the time budget and remaining strong through size 12. In contrast, the learning-based methods stay far below perfect accuracy in this regime, with even the best pipeline reaching only about 0.8 on the easiest boards.

At the same time, RL consistently improves over SFT alone across almost the entire range, showing that imitation is useful but insufficient by itself. The gain becomes especially important on medium and large boards: for example, RL raises solve rate from 0.275 to 0.450 at size 14, and from 0.085 to 0.285 at size 20.

Most importantly, once the board is large enough, the learned policy overtakes the time-limited symbolic solver. Starting around $n = 16$, SFT+RL surpasses the solver and the gap becomes substantial at larger sizes; at size 18, for instance, the solver achieves 0.095 while SFT+RL reaches 0.395. This highlights the trade-off between the two paradigms: symbolic search is highly reliable when the search space is manageable, whereas the learned policy is less accurate on easy instances but scales more gracefully because its inference cost is essentially fixed after training.

Overall, RL does not replace symbolic search on small Hitori boards, but it becomes increasingly valuable as size grows. Combined with SFT, it turns an imperfect supervised policy into a more scalable amortized solver that can outperform a strict-budget symbolic baseline on sufficiently large instances.

4 Conclusion

In this project, we developed a deep learning based Hitori solver by combining cold-start supervised fine-tuning, Maskable PPO, a U-Net style policy network, variable-size adaptation, and shaped reward. Together, these components form a practical reinforcement-learning pipeline for a highly constrained combinatorial puzzle. Empirically, the resulting model is able to learn nontrivial Hitori-solving behavior across a wide range of board sizes, and in the large-size regime it even surpasses a search-based symbolic solver under a fixed inference-time budget.

At the same time, our results also reveal clear limitations. On small boards, where symbolic reasoning is comparatively easy, the learned policy cannot approach perfect accuracy. Even for very small instances, its solve rate remains far below 1.0, suggesting that the current pipeline still behaves as an approximate amortized solver rather than a fully reliable decision procedure. In contrast, on larger boards the learning-based approach becomes more competitive because its inference cost remains essentially fixed, while the effectiveness of time-limited search degrades rapidly.

These observations suggest that the current method is promising but far from the final word. There are likely stronger reinforcement-learning approaches for this problem than the current PPO-based pipeline. Even within the PPO framework, better optimization strategies, improved architectures, or more effective use of demonstrations may still yield substantial gains. More importantly, Hitori appears to be a particularly natural candidate for search-augmented reinforcement learning. Methods in the spirit of AlphaZero [13], which combine a learned policy-value network with Monte Carlo Tree Search, may be a more appropriate long-term direction than pure PPO for such a structured puzzle domain.

Due to time constraints, we were not able to explore these alternatives further in this project. Nevertheless, based on the empirical behavior observed here, we believe that there should exist practical methods for pushing solve rates on much larger boards—potentially board sizes in the tens—to above 90% without requiring prohibitively large computational resources. We hope this project can serve as a useful starting point for future work on combining reinforcement learning, supervised initialization, and structured search for logic puzzle solving.

References

[1] Nikoli. Hitori. `https://www.nikoli.co.jp/en/puzzles/hitori/`, 2026. Accessed: 2026-05-03.

[2] Maria Leonor Pacheco, Fabio Somenzi, Dananjay Srinivas, and Ashutosh Trivedi. Explaining hitori puzzles: Neurosymbolic proof staging for sequential decisions. *arXiv preprint arXiv:2508.14294*, 2025.

[3] Sappho de Nooij. Solving hitori: Applying answer set programming to hitori. Master's thesis, Delft University of Technology, 2026.

[4] Matthias Gander and Christian Hofer. Hitori solver. Master's thesis, University of Innsbruck, 2006.

[5] R. J. Rietdijk. Solving hitori puzzles using satisfiability modulo theories. Master's thesis, Delft University of Technology, 2026.

[6] S. van Luenen. Eating soup with a fork: Solving hitori with integer linear programming. Master's thesis, Delft University of Technology, 2026.

[7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[8] Shengyi Huang and Santiago Ontañón. A closer look at invalid action masking in policy gradient algorithms. In *Proceedings of the International Florida Artificial Intelligence Research Society Conference*, volume 35, 2022.

[9] Vibhu Agarwal. hitori-gym: A gymnasium environment for the hitori puzzle with action masking. `https://github.com/Vibhu-Agarwal/hitori-gym`, 2025. GitHub repository.

[10] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention*, pages 234–241. Springer, 2015.

[11] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 278–287. Morgan Kaufmann, 1999.

[12] Philip Lugt. hitori-solver. `https://github.com/philiplugt/hitori-solver`, 2024. GitHub repository.

[13] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

Algorithm 1 Brute-force Hitori solver

Require: puzzle grid P

$counter \leftarrow 0$
$nodes \leftarrow [(0, 0)]$
$states \leftarrow [\text{LOADPUZZLE}(P)]$
while $nodes$ is not empty **do**
    $(i, j) \leftarrow nodes.\text{POP}$
    $state \leftarrow states.\text{POP}$
    if CHECKSOLUTION($state$) **then**
        return $(counter,\ state.puzzle)$
    end if
    if $i \geq$ number of rows **then**
        continue
    end if
    if $state.domain[i][j] =$ `V` **then**
        push (NEXTCELL(i, j)) and $state$
        continue
    end if
    $white\_state \leftarrow \text{COPY}(state)$
    $white\_state.domain[i][j] \leftarrow$ `W`
    $counter \leftarrow counter + 1$
    push (NEXTCELL(i, j)) and $white\_state$
    if BLACKALLOWED($state, i, j$) **then**
        $black\_state \leftarrow \text{COPY}(state)$
        $black\_state.puzzle[i][j] \leftarrow$ `B`
        $black\_state.domain[i][j] \leftarrow$ `B`
        $counter \leftarrow counter + 1$
        if TESTWHITECONNECTED($black\_state.domain,\ black\_state.edges$) **then**
            push (NEXTCELL(i, j)) and $black\_state$
        end if
    end if
end while
return $(counter,$ `None`$)$

A Reference Solver Algorithm

For completeness, we summarize the search procedure used in the reference GitHub Hitori solver that we used as a symbolic baseline and as a source of supervision. The implementation maintains an explicit depth-first search stack over partial board assignments. Each state contains the current puzzle, a per-cell domain, and the auxiliary graph structure needed for connectivity checks. Cells are visited in row-major order; thus, although the original code comments mention CSP/MRV, the main source of efficiency in the provided implementation is forward checking and early pruning rather than dynamic variable reordering.

A.1 Brute-Force Generate-and-Test Solver

The brute-force solver explores white and black assignments for each cell, subject only to local black-cell legality and a global white-connectivity check after black assignments. Cells whose initial domain is already fixed to `V` (meaning the value is unique and therefore must stay white) are skipped directly.

A.2 Smart Solver with Forward Checking

The smart solver keeps the same depth-first skeleton but adds several CSP-style pruning operations. Before branching, it applies a deterministic local simplification routine. When assigning a cell to white, it performs forward checking for the duplicate-removal rule; when assigning a cell to black, it

Algorithm 2 Smart Hitori solver with forward checking

Require: puzzle grid P

$counter \leftarrow 0$
$nodes \leftarrow [(0, 0)]$
$states \leftarrow [\text{LOADPUZZLE}(P)]$
while $nodes$ is not empty **do**
    $(i, j) \leftarrow nodes.\text{POP}$
    $state \leftarrow states.\text{POP}$
    if CHECKSOLUTION($state$) **then**
        return $(counter,\ state.puzzle)$
    end if
    if $i \geq$ number of rows **then**
        continue
    end if
    $state \leftarrow \text{CELLSURROUNDED}(state, i, j)$
    if $state =$ `None` **then**
        continue
    end if
    for $d \in state.domain[i][j]$ **do**
        if $d =$ `V` **then**
            push $(\text{NEXTCELL}(i, j))$ and $state$
            continue
        end if
        if $d =$ `W` **then**
            $white\_state \leftarrow \text{COPY}(state)$
            $white\_state.domain[i][j] \leftarrow$ `W`
            $counter \leftarrow counter + 1$
            if FCWHITE($white\_state, i, j$) **then**
                push $(\text{NEXTCELL}(i, j))$ and $white\_state$
            end if
        end if
        if $d =$ `B` **and** BLACKALLOWED($state, i, j$) **then**
            $black\_state \leftarrow \text{COPY}(state)$
            $black\_state.puzzle[i][j] \leftarrow$ `B`
            $black\_state.domain[i][j] \leftarrow$ `B`
            $counter \leftarrow counter + 1$
            if TESTWHITECONNECTED($black\_state.domain$, $black\_state.edges$) **then**
                $black\_state \leftarrow \text{FCBLACK}(black\_state, i, j)$
                if $black\_state \neq$ `None` **then**
                    push $(\text{NEXTCELL}(i, j))$ and $black\_state$
                end if
            end if
        end if
    end for
end while
return $(counter,$ `None`$)$

first checks white-cell connectivity and then propagates the non-adjacent-black constraint. In practice, these pruning steps are what make the smart solver much more efficient than naive generate-and-test.

A.3 Role of the Helper Routines

Algorithm 1 and Algorithm 2 rely on a small set of helper routines that encode the Hitori constraints. `LoadPuzzle` initializes the domain and connectivity bookkeeping for a fresh puzzle. `BlackAllowed` checks whether shading a cell immediately violates the "no adjacent black cells" rule. `TestWhiteConnected` rejects black assignments that disconnect the remaining white-cell graph. `FCWhite` propagates the duplicate-removal constraint after assigning a cell to white, while `FCBlack`

propagates consequences of assigning a cell to black. Finally, `CellSurrounded` performs a local deterministic simplification before branching. Together, these routines turn a plain DFS over cell colors into a substantially more effective constraint-based search procedure.